

Build Task — Project 2: Tool-Using Agent — Hints & Solution

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python — how a real agent codebase would actually write it). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — What you're building, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)
- [Solution](#)

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — What you're building, and the exact pieces

Basic Version

You're building 1 agent, made of everything the 5 exercises above already gave you: the loop from `build_react_loop`, the "let it answer directly" check from `no_tool_needed`, the step limit from `infinite_loop_cost`, and 3+ tools reused from `06_tools_function_calling`.

The 3 files that matter most: - `tools.py` — the same `@tool`-decorated functions from Doc06, unchanged. - `agent.py` — `run_agent(task, max_iterations) -> AgentResult`, the loop itself. - `test_agent.py` — the 4 Test Cases from the table above, each as a real test.

Things to use: - A `while / for` loop with a step counter (from `build_react_loop`). - `try / except` around every tool call, so one failing tool doesn't crash the whole agent. - A `MaxIterationsExceeded` error, raised — not silently swallowed — when the limit is hit. - A `steps` list, recording every round, so the agent's run is debuggable afterward.

Intermediate Version

Think of `agent.py` as 3 separate concerns living in one loop: **deciding** (call the model, let it choose a tool or answer directly), **doing** (run whichever tool it picked, catching any failure), and **remembering** (log the step, append to the message history). Keep these 3 ideas straight in your head even though they live in the same function — most bugs in an agent loop come from mixing them up, like forgetting to log a step, or letting a tool's exception skip the logging entirely.

- `run_agent(task: str, max_iterations: int) -> AgentResult` — `AgentResult` should hold at least `final_answer` and `steps`.
- Reuse your Doc06 tools by importing them, not rewriting them — `from tools import get_weather, convert_currency, search_docs` (or whatever your 3+ tools are named).
- The tool-failure requirement means at least 1 of your tools needs a way to fail *on purpose*, in a controlled way, for testing — think about how you'd make a tool fail once and then succeed, without permanently breaking it for other tests.
- `MaxIterationsExceeded` should carry the partial `steps` log with it when raised — a bare error message with no context is much harder to debug than one carrying exactly what happened before it gave up.
- Every tool call's arguments must pass Pydantic checking before the tool actually runs — a real constraint from the README, not optional polish. Validate with a Pydantic model *before* calling the tool function, so a malformed call never reaches it.

The real design question here isn't "does the loop work" — you already built that. It's **how do you make a tool fail on a real run without making your test suite fragile or your production tool permanently broken?** A tool that always fails isn't useful for proving recovery (the agent never gets a chance to succeed); a tool that never fails can't prove the recovery path exists at all. The answer that scales: a tool wrapped with a small, controllable amount of state — a counter that fails the tool's first call and succeeds on every call after that, reset between test runs. This same pattern is what lets `test_agent.py`'s "fail once then succeed" row in the Test Cases table be a real, repeatable, automated test instead of something you have to trigger by hand and watch.

A second design question: **where does step-logging live so every code path goes through it?** If you log a step only in the "tool call succeeded" branch, a failed tool call's attempt never gets recorded — which defeats the entire point of the log for debugging exactly the runs that went wrong. Log the attempt and its outcome (success or the caught error) from one place, not scattered across branches.

Difference between Basic and Intermediate: Basic names the files, the reused pieces, and the exact tools you'll need. Intermediate separates the loop into deciding/doing/remembering, adds the type contract real Python expects, and answers the 2 harder design questions underneath the requirements — how to make a tool controllably, repeatably failable for testing, and how to make sure every code path (success *and* failure) actually gets logged — both of which only bite once you try to write `test_agent.py` for real.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
tools.py:
    reused, unchanged, from 06_tools_function_calling

agent.py:
    make a MaxIterationsExceeded error, it's a kind of Exception
    make an AgentResult holder: final_answer, steps

    function run_agent(task, max_iterations):
        messages = [starting task]
        steps = []

        repeat up to max_iterations times:
            ask the model, with all tools available, using messages so far
            if the model asked for a tool:
                try to run it
                if it works: record the result
                if it fails: record the error, don't crash
                add everything to messages and steps
            else:
                return AgentResult(final_answer, steps)

        raise MaxIterationsExceeded, with the partial steps attached
```

The trickiest part — running a tool without letting a failure crash the loop:

```
def run_tool(call, tools_by_name):
    function = tools_by_name.get(call.function.name)
    try:
        args = json.loads(call.function.arguments)
        result = function(**args)
        return str(result), None
    except Exception as e:
        return None, str(e)
```

Expected output if you call this with a working tool and good arguments: a `(result, None)` tuple — the error slot is `None` because nothing went wrong.

Intermediate Version

```
agent.py:
    class MaxIterationsExceeded(Exception): carries the partial steps list

    @dataclass
    class AgentResult:
        final_answer: str
        steps: list[dict]

    function run_tool(call, tools_by_name) -> (str | None, str | None):
        look up the function by name
        try: run it, return (result_as_string, None)
        except Exception as e: return (None, str(e))

    function run_agent(task: str, max_iterations: int) -> AgentResult:
        messages = [{"role": "user", "content": task}]
        steps = []

        for step in range(max_iterations):
            response = call the model with messages and all tool schemas
            message = response.choices[0].message

            if message.tool_calls:
                messages.append(message)
                for call in message.tool_calls:
                    result, error = run_tool(call, tools_by_name)
                    observation = result if error is None else f"Error: {error}"
                    messages.append({
                        "role": "tool", "tool_call_id": call.id,
                        "content": observation,
                    })
                    steps.append({"step": step, "tool": call.function.name,
                                "arguments": call.function.arguments,
                                "result": result, "error": error})
            else:
                return AgentResult(final_answer=message.content, steps=steps)

        raise MaxIterationsExceeded(f"No answer after {max_iterations} steps", st
```

Turn this into real code, then write the controllable-failure tool yourself before checking the Solution.

Tests need a tool that fails exactly once, then works normally, so the "recovers from a failure" test case is real and repeatable:

```
class FlakyToolState:
    call_count = 0

function flaky_search_docs(query):
    FlakyToolState.call_count += 1
    if FlakyToolState.call_count == 1:
        raise ConnectionError("search index temporarily unavailable")
    return search_docs(query)  # the real Doc06 tool, once past the first call
```

```
class FlakyToolState:
    def __init__(self) -> None:
        self.call_count = 0

def make_flaky_search_docs(state: FlakyToolState):
    def flaky_search_docs(query: str) -> str:
        state.call_count += 1
        if state.call_count == 1:
            raise ConnectionError("search index temporarily unavailable")
        return search_docs(query)
    return flaky_search_docs
```

`make_flaky_search_docs` returns a *fresh* flaky function tied to a fresh `FlakyToolState()` each time it's called — so every test that needs "fails once, then works" gets its own independent counter, instead of tests accidentally sharing state and interfering with each other.

Fill in the rest of `run_agent()` and `run_tool()` yourself, wire `flaky_search_docs` in as one of your 3+ tools for a test run, then compare your finished version against the [Solution](#).

Difference between Basic and Intermediate: Basic's pseudocode and near-complete code prove the core idea — catch, don't crash — works at all. Intermediate is the same shape, fully typed, with the log recording both the result and the error side by side so nothing about a failed attempt gets lost, plus the piece that makes the Build Task's failure requirement testable rather than just demonstrable-once-by-hand: a tool wrapped in its own small piece of state, so "fails once then succeeds" is a real, repeatable setup any test can use.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Solution

Every code block below shows the exact output you'd see if you ran it. Read both depths — they're not "wrong, right," they're 2 real, valid ways to solve the same problem, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way

Story — `agent.py` : this is Project 2's Worker — every practice exercise above (the loop, the no-tool check, the step limit) folds into this one file. **If not:** Project 2 would be the first place any of these pieces ever had to work together, with no smaller version to trust.

```
# agent.py
import json

class MaxIterationsExceeded(Exception):
    def __init__(self, message, steps):
        self.steps = steps
        super().__init__(message)

class AgentResult:
    def __init__(self, final_answer, steps):
        self.final_answer = final_answer
        self.steps = steps

def run_tool(call, tools_by_name):
    function = tools_by_name.get(call.function.name)
    if function is None:
        return None, f"unknown tool: {call.function.name}"
    try:
        args = json.loads(call.function.arguments)
        result = function(**args)
        return str(result), None
    except Exception as e:
        return None, str(e)
```

```
def run_agent(task, max_iterations, tools_by_name, tool_schemas, client):
    messages = [{"role": "user", "content": task}]
    steps = []

    for step in range(max_iterations):
        response = client.chat.completions.create(
            model="gpt-4o-mini", messages=messages, tools=tool_schemas,
        )
        message = response.choices[0].message

        if message.tool_calls:
            messages.append(message)
            for call in message.tool_calls:
                result, error = run_tool(call, tools_by_name)
                observation = result if error is None else "Error: " + error
                messages.append({
                    "role": "tool", "tool_call_id": call.id, "content": observation
                })
                steps.append({
                    "step": step,
                    "tool": call.function.name,
                    "arguments": call.function.arguments,
                    "result": result,
                    "error": error,
                })
            else:
                return AgentResult(message.content, steps)

    message = "No answer after " + str(max_iterations) + " steps"
    raise MaxIterationsExceeded(message, steps)
```

```
# main.py
from tools import get_weather, celsius_to_fahrenheit, search_docs
from agent import run_agent

TOOLS_BY_NAME = {
    "get_weather": get_weather,
    "celsius_to_fahrenheit": celsius_to_fahrenheit,
    "search_docs": search_docs,
}

task = input("What do you need? ")
result = run_agent(
    task, max_iterations=5, tools_by_name=TOOLS_BY_NAME,
```

```

    tool_schemas=ALL_SCHEMAS, client=client,
)
print(result.final_answer)
for step in result.steps:
    print(step)

```

Expected output for "What's the weather in Paris, in Fahrenheit?":

```

It's 64.4°F in Paris.
{'step': 0, 'tool': 'get_weather', 'arguments': '{"city": "Paris"}',
 'result': '18', 'error': None}
{'step': 1, 'tool': 'celsius_to_fahrenheit',
 'arguments': '{"celsius": 18}', 'result': '64.4', 'error': None}

```

(the 2 `step` dicts above are shown wrapped onto two lines each just to fit the page — each is really one line of output) This meets every Build Task requirement. It's missing type hints and a real `@dataclass`, and passes `tools_by_name` / `tool_schemas` / `client` as plain arguments instead of module-level setup — both fine for a first working version.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — dataclasses, type hints, a real dispatch table

Story — `agent.py` (Intermediate): Basic proved the loop works; this version makes it safe to build on — a typed `AgentResult` instead of a loose dict, and a real `tools_by_name` dispatch table instead of an `if/elif` chain that grows one branch per tool forever. **If not:** every new tool in Project 2 would mean editing `run_tool()`'s branching logic instead of just adding one line to a dictionary — and a typo in a field name would fail silently instead of being caught by the type checker.

```
import json
from dataclasses import dataclass, field
from openai import OpenAI

client = OpenAI()

class MaxIterationsExceeded(Exception):
    def __init__(self, message: str, steps: list[dict]) -> None:
        # why: callers need the partial step log to debug why it looped
        self.steps = steps
        super().__init__(message)

@dataclass
class AgentResult:
    final_answer: str
    # why: mutable default needs a factory, not `=[]`
    steps: list[dict] = field(default_factory=list)

def run_tool(call, tools_by_name: dict) -> tuple[str | None, str | None]:
    function = tools_by_name.get(call.function.name)
    if function is None:
        # how: model hallucinated a tool name
        return None, f"unknown tool: {call.function.name}"
    try:
        args = json.loads(call.function.arguments)
        result = function(**args)
        return str(result), None
```

```

except Exception as e:
    # why: any tool failure becomes an Observation, never a crash
    return None, str(e)

def run_agent(
    task: str,
    max_iterations: int,
    tools_by_name: dict,
    tool_schemas: list[dict],
) -> AgentResult:
    messages: list[dict] = [{"role": "user", "content": task}]
    steps: list[dict] = []

    for step in range(max_iterations):
        response = client.chat.completions.create(
            model="gpt-4o-mini", messages=messages, tools=tool_schemas,
        )
        message = response.choices[0].message

        if message.tool_calls:
            messages.append(message)
            for call in message.tool_calls:
                result, error = run_tool(call, tools_by_name)
                observation = result if error is None else f"Error: {error}"
                messages.append({
                    "role": "tool", "tool_call_id": call.id, "content": observati
                })
                steps.append({
                    "step": step,
                    "tool": call.function.name,
                    "arguments": call.function.arguments,
                    "result": result,
                    "error": error,
                })
            else:
                # when: no tool_calls means the model is done – the loop's only exit
                return AgentResult(final_answer=message.content, steps=steps)

    message = f"No answer after {max_iterations} steps"
    raise MaxIterationsExceeded(message, steps)

```

main.py

```

from tools import get_weather, celsius_to_fahrenheit, search_docs
from agent import run_agent, MaxIterationsExceeded

```

```

TOOLS_BY_NAME = {
    "get_weather": get_weather,
    "celsius_to_fahrenheit": celsius_to_fahrenheit,
    "search_docs": search_docs,
}

def main() -> None:
    task = input("What do you need? ")
    try:
        result = run_agent(
            task, max_iterations=5, tools_by_name=TOOLS_BY_NAME,
            tool_schemas=ALL_SCHEMAS,
        )
    except MaxIterationsExceeded as e:
        print(f"Gave up: {e}")
        for step in e.steps:
            print(step)
        return

    print(result.final_answer)
    for step in result.steps:
        print(step)

if __name__ == "__main__":
    main()

```

Expected output for a task designed to loop forever, with `max_iterations=3`:

```

Gave up: No answer after 3 steps
{'step': 0, 'tool': 'search_docs', 'arguments': '{"query": "..."}',
 'result': 'no matches found', 'error': None}
{'step': 1, 'tool': 'search_docs', 'arguments': '{"query": "..."}',
 'result': 'no matches found', 'error': None}
{'step': 2, 'tool': 'search_docs', 'arguments': '{"query": "..."}',
 'result': 'no matches found', 'error': None}

```

(each `step` dict above is shown wrapped onto two lines just to fit the page — really one line of output each)

Approach 2 — a real logger instead of `print`, per Core Concepts' step-log requirement

Story — `agent.py` (Approach 2): Core Concepts requires a step log you can actually find later; `print()` output scrolls off and disappears the moment the terminal closes, a real logger's output

doesn't. **If not:** the one time you'd need to know why a Project 2 run looped 5 times last night, the evidence would already be gone.

```
import logging

# why: named logger, not root – callers can filter just this one
logger = logging.getLogger("agent")

def run_agent(task, max_iterations, tools_by_name, tool_schemas):
    messages = [{"role": "user", "content": task}]
    steps = []

    for step in range(max_iterations):
        response = client.chat.completions.create(
            model="gpt-4o-mini", messages=messages, tools=tool_schemas,
        )
        message = response.choices[0].message

        if message.tool_calls:
            messages.append(message)
            for call in message.tool_calls:
                result, error = run_tool(call, tools_by_name)
                observation = result if error is None else f"Error: {error}"
                messages.append({
                    "role": "tool", "tool_call_id": call.id, "content": observation
                })
                step_record = {
                    "step": step, "tool": call.function.name,
                    "arguments": call.function.arguments,
                    "result": result, "error": error,
                }
                steps.append(step_record)
                if error is None:
                    logger.info(
                        "step %s: %s -> %s", step, call.function.name, result,
                    )
                else:
                    # how: warning, not error – a recovered failure isn't fatal
                    logger.warning(
                        "step %s: %s failed -> %s",
                        step, call.function.name, error,
                    )
            else:
                return AgentResult(final_answer=message.content, steps=steps)
```

```
message = f"No answer after {max_iterations} steps"
raise MaxIterationsExceeded(message, steps)
```

Difference from Approach 1: Approach 1's `steps` list is the log — fine for a short script you run and read once. Approach 2 adds real `logging` calls alongside the same `steps` list, so a failed tool call is visibly a `warning` -level line in your terminal or log file as it happens, not just something you'd notice later by scanning the returned list.

Difference from Basic: both Intermediate approaches add full type hints and a real `@dataclass` for `AgentResult`, and pass tools/schemas as explicit arguments instead of relying on globals. Approach 2 additionally logs every step through Python's `logging` module as it happens, distinguishing a success from a caught tool failure at the log level, not just in the data.

Approach 3 — a controllably-flaky tool, and the full `test_agent.py` it enables

Story — `tools.py` addition + `test_agent.py`: the README's 4 Test Cases include "recovers from a tool failure," but a real tool fails at random — you can't write a repeatable automated test around something that only sometimes happens. `FlakyToolState` makes the failure happen on command, every time. **If not:** the failure-recovery Test Case would stay something you check once by hand and hope keeps working, instead of a real test that runs every time you change `agent.py`.

`tools.py` (addition, alongside the real reused Doc06 tools)

```
class FlakyToolState:
    def __init__(self) -> None:
        # why: shared mutable state the closure below reads and updates
        self.call_count = 0

def make_flaky_search_docs(state: FlakyToolState):
    def flaky_search_docs(query: str) -> str:
        state.call_count += 1
        if state.call_count == 1:
            # how: fails on call 1 only, so a retry always succeeds
            raise ConnectionError("search index temporarily unavailable")
        return search_docs(query)
    return flaky_search_docs
```

`test_agent.py`

```
from agent import run_agent, MaxIterationsExceeded
from tools import (
    get_weather, celsius_to_fahrenheit, search_docs,
    FlakyToolState, make_flaky_search_docs,
)
```

```

def test_clear_task_uses_the_right_tool():
    tools_by_name = {
        "get_weather": get_weather,
        "celsius_to_fahrenheit": celsius_to_fahrenheit,
    }
    result = run_agent(
        "What's the weather in Paris?", 5, tools_by_name, WEATHER_SCHEMAS,
    )
    assert len(result.steps) >= 1
    assert result.steps[0]["tool"] == "get_weather"

def test_no_tool_needed_for_general_knowledge():
    tools_by_name = {
        "get_weather": get_weather,
        "celsius_to_fahrenheit": celsius_to_fahrenheit,
    }
    result = run_agent(
        "What's the capital of France?", 5, tools_by_name, WEATHER_SCHEMAS,
    )
    assert len(result.steps) == 0

def test_flaky_tool_recovers_on_retry():
    # how: fresh state per test – call_count starts at 0
    state = FlakyToolState()
    flaky_tool = make_flaky_search_docs(state)
    tools_by_name = {"search_docs": flaky_tool}

    result = run_agent(
        "Search the docs for 'refund policy'", 5, tools_by_name, SEARCH_SCHEMAS,
    )

    failed_steps = [s for s in result.steps if s["error"] is not None]
    succeeded_steps = [s for s in result.steps if s["error"] is None]
    assert len(failed_steps) >= 1
    assert len(succeeded_steps) >= 1
    assert result.final_answer # it recovered and actually answered

def test_step_limit_is_enforced_and_raises():
    tools_by_name = {"search_docs": search_docs}
    try:
        run_agent(
            "a task designed to never be satisfied",

```

```

        3, tools_by_name, SEARCH_SCHEMAS,
    )
    assert False, "expected MaxIterationsExceeded"
except MaxIterationsExceeded as e:
    # why: the partial log must survive the raise
    assert len(e.steps) == 3

```

Expected output when run with pytest:

4 passed

Notice `test_flaky_tool_recovers_on_retry` filters `result.steps` into `failed_steps` and `succeeded_steps` rather than asserting an exact count or order — it's checking the *shape* of recovery (at least 1 failure, at least 1 success, and a real final answer), not a brittle exact step count that would break the moment the model's retry behavior shifts slightly.

Approach 4 — validating tool arguments with Pydantic before the tool ever runs

Story — `run_tool()` with Pydantic: the README's own Constraint says every tool call's arguments must pass Pydantic checking *before* the tool runs — not "the tool should handle bad input gracefully," but the check has to happen earlier than that, as a separate step. **If not:** a malformed argument (a string where a number was expected) would reach the tool function itself, and whatever it does with bad input — crash, or silently produce a wrong answer — would be Project 2's actual behavior, not a deliberate, caught, reported error.

```

from pydantic import BaseModel, ValidationError

class SearchDocsArgs(BaseModel):
    query: str

def run_tool(call, tools_by_name, arg_models):
    function = tools_by_name.get(call.function.name)
    arg_model = arg_models.get(call.function.name)
    if function is None:
        return None, f"unknown tool: {call.function.name}"
    try:
        raw_args = json.loads(call.function.arguments)
        if arg_model is not None:
            # why: validate BEFORE calling function — this is what "before the tool
            # actually runs" in the Build Task's constraint means, literally
            validated = arg_model(**raw_args)
            result = function(**validated.model_dump())
        else:

```

```

        result = function(**raw_args)
        return str(result), None
    except ValidationError as e:
        return None, f"invalid arguments: {e}"
    except Exception as e:
        return None, str(e)

```

Expected behavior if the model calls `search_docs` with `{"query": 123}` (a number, not a string): Pydantic's `SearchDocsArgs(**raw_args)` raises `ValidationError` before `search_docs` itself is ever called — the loop's next Observation is `"Error: invalid arguments: ..."`, and the tool function never runs with bad input. This is what the Build Task's constraint — "every tool call's arguments must pass Pydantic checking before the tool actually runs" — means literally: the check happens *before* the call, not as a side effect of the call failing.

Difference from Approach 1/2: Approach 1/2's `run_tool()` catches whatever exception the tool itself happens to raise — including a `TypeError` from bad arguments, but only *after* the tool already started running with them. Approach 3 adds the piece that makes the "recovers from a failure" requirement genuinely testable, not just plausible. Approach 4 moves argument validation earlier, catching a malformed call before the tool function runs at all, matching the Build Task's constraint precisely instead of relying on whatever exception the tool happens to throw.

Which one should you actually build? Approach 1's shape is what most of `agent.py` should look like — typed, a real dispatch table, errors caught and logged per step. Layer in Approach 4's Pydantic validation for every tool (this is a hard constraint, not optional), and Approach 3's `FlakyToolState` pattern in `test_agent.py` so the failure-recovery Test Case is a real, repeatable, automated test — not something you trigger by hand once and never check again.